📐 Phase 0 — Étape 0.1 : Remise à niveau Mathématiques

> **Durée estimée** : ~25h réparties sur 6 à 8 semaines (à 3h/sem)
> **Approche** : Théorie + pratique en parallèle
> **Objectif** : Avoir les bases maths indispensables pour la Data Science et le ML

📑 Sommaire

1. Algèbre linéaire
2. Statistiques descriptives
3. Probabilités
4. Calcul différentiel (bases)
5. Planning suggéré
6. Ressources

1. Algèbre linéaire

> Pourquoi ? Les données sont des matrices. Chaque algorithme ML manipule des vecteurs et matrices en permanence.

🔑 Notions fondamentales

Scalaire, Vecteur, Matrice

- **Scalaire** : un simple nombre → `5`, `3.14`
- **Vecteur** : une liste ordonnée de nombres → `[1, 2, 3]` (une ligne ou une colonne)
- **Matrice** : tableau 2D de nombres → une image est une matrice de pixels, un dataset est une matrice lignes × colonnes

```
Vecteur v = [1, 2, 3]        Matrice A (2×3) :
                              | 1  2  3 |
                              | 4  5  6 |
```

Opérations sur les vecteurs

- **Addition** : `[1,2] + [3,4] = [4,6]` → on additionne terme à terme
- **Multiplication par un scalaire** : `2 × [1,2] = [2,4]`
- **Produit scalaire (dot product)** : `[1,2,3] · [4,5,6] = 1×4 + 2×5 + 3×6 = 32`
  - 💡 *Utilisé partout en ML pour calculer des similarités et des prédictions*

Opérations sur les matrices

- **Addition** : même dimension, terme à terme
- **Transposition** : inverser lignes et colonnes → une matrice (2×3) devient (3×2)

- **Multiplication matricielle** : `A(m×n) × B(n×p) = C(m×p)`
  - ⚠️ L'ordre compte : `A×B ≠ B×A` en général

```
A = | 1  2 |    B = | 5  6 |    A×B = | 1×5+2×7  1×6+2×8 | = | 19  22 |
    | 3  4 |        | 7  8 |          | 3×5+4×7  3×6+4×8 |   | 43  50 |
```

✏️ Exercices pratiques

Ex 1.1 — Calculer à la main :

- `[2, 4, 6] · [1, 0, 3]` = ?
- Transposée de `[[1,2,3],[4,5,6]]` = ?

Ex 1.2 — En Python (à faire dans un notebook) :

```python
import numpy as np

v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])

print("Produit scalaire :", np.dot(v1, v2))

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("Produit matriciel :\n", np.matmul(A, B))
print("Transposée de A :\n", A.T)
```

2. Statistiques descriptives

> Pourquoi ? Avant tout modèle ML, on doit comprendre et résumer ses données.

🔑 Notions fondamentales

Mesures de tendance centrale

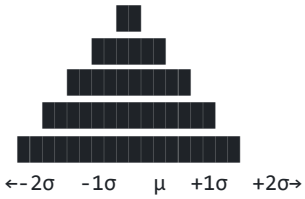
- **Moyenne (mean)** : somme des valeurs / nombre de valeurs
  - Ex : `[2, 4, 4, 6] → (2+4+4+6)/4 = 4`
- **Médiane** : valeur du milieu une fois les données triées
  - Ex : `[1, 3, 5, 7, 9] → médiane = 5`
  - 💡 *Plus robuste aux valeurs extrêmes (outliers) que la moyenne*
- **Mode** : valeur la plus fréquente

Mesures de dispersion

- **Variance (σ^2)** : mesure l'écart moyen au carré par rapport à la moyenne
  - $\sigma^2 = \Sigma(x_i - \mu)^2 / n$
- **Écart-type (σ)** : racine carrée de la variance → même unité que les données
  - Un petit σ = données serrées autour de la moyenne
  - Un grand σ = données très dispersées

Distribution & histogramme

- Un **histogramme** montre la répartition des valeurs
- **Distribution normale (courbe en cloche)** : très fréquente en nature et en data
  - 68% des données sont à ±1σ de la moyenne
  - 95% à ±2σ
  - 99.7% à ±3σ



📝 Exercices pratiques

Ex 2.1 — À la main :

- Dataset : `[10, 12, 12, 13, 12, 11, 14, 13, 15, 10]`
  - Calculer la moyenne, la médiane, le mode
  - Calculer la variance et l'écart-type

Ex 2.2 — En Python :

```python
import numpy as np
import matplotlib.pyplot as plt

data = [10, 12, 12, 13, 12, 11, 14, 13, 15, 10]

print("Moyenne :", np.mean(data))
print("Médiane :", np.median(data))
print("Écart-type :", np.std(data))
print("Variance :", np.var(data))

# Visualiser la distribution
plt.hist(data, bins=5, color='steelblue', edgecolor='white')
plt.title("Distribution des données")
plt.xlabel("Valeur")
plt.ylabel("Fréquence")
plt.show()
```

3. Probabilités

> Pourquoi ? Les modèles ML raisonnent en termes de probabilités (prédire = estimer une probabilité).

🔑 Notions fondamentales

Concepts de base

- **Probabilité** : nombre entre 0 et 1 exprimant la chance qu'un événement se produise

  - `P(A) = 0` → impossible | `P(A) = 1` → certain
- **Événements indépendants** : `P(A et B) = P(A) × P(B)`
- **Probabilité conditionnelle** : `P(A | B)` = probabilité de A sachant que B s'est produit

Loi des grands nombres

> Plus on répète une expérience, plus la fréquence observée se rapproche de la probabilité théorique.

- Lancer une pièce 10 fois → peut donner 7 faces / 3 piles
- Lancer 10 000 fois → ~50% faces, ~50% piles

Loi normale (rappel)

- Paramétrée par sa moyenne `μ` et son écart-type `σ`
- Notation : `X ~ N(μ, σ²)`
- Fondamentale : beaucoup de phénomènes naturels et d'erreurs suivent cette loi

Théorème de Bayes *(notion clé en ML)*

```
P(A | B) = P(B | A) × P(A) / P(B)
```

- **Intuition** : mettre à jour sa croyance sur A quand on observe B
- 💡 *Ex : probabilité qu'un mail soit spam, sachant qu'il contient le mot "gratuit"*

✏️ Exercices pratiques

Ex 3.1 — À la main :

- Un dé à 6 faces. Quelle est la probabilité d'obtenir un nombre pair ?
- On tire 2 cartes d'un jeu (sans remise). Prob d'obtenir 2 as ?

Ex 3.2 — En Python (simuler la loi des grands nombres) :

```python
import numpy as np
import matplotlib.pyplot as plt

# Simuler des lancers de pièce
n_lancers = [10, 100, 1000, 10000]
for n in n_lancers:
    resultats = np.random.randint(0, 2, n)  # 0=pile, 1=face
    freq_face = np.sum(resultats) / n
    print(f"n={n:6d} → fréquence faces : {freq_face:.3f}")
```

4. Calcul différentiel

> Pourquoi ? L'entraînement d'un modèle ML = minimiser une erreur. Pour ça, on utilise le gradient (dérivée généralisée).

🔑 Notions fondamentales

La dérivée : qu'est-ce que c'est ?

- La dérivée `f'(x)` mesure la **pente** (taux de variation) d'une fonction en un point
- Si `f'(x) > 0` → la fonction monte en x
- Si `f'(x) < 0` → la fonction descend en x
- Si `f'(x) = 0` → c'est un minimum ou un maximum local

```
f(x) = x²
     |    *
     |  *   *
     |*       *
─────┼──────────→ x
     0

f'(x) = 2x → zéro en x=0 (minimum)
```

Dérivées usuelles (à connaître par cœur)

Fonction	Dérivée
`f(x) = c` (constante)	`f'(x) = 0`
`f(x) = x`	`f'(x) = 1`
`f(x) = x²`	`f'(x) = 2x`
`f(x) = xⁿ`	`f'(x) = n·xⁿ⁻¹`
`f(x) = eˣ`	`f'(x) = eˣ`
`f(x) = ln(x)`	`f'(x) = 1/x`

Le gradient (généralisation en plusieurs dimensions)

- Pour une fonction de plusieurs variables `f(x, y)`, le gradient est le vecteur des dérivées partielles :
  - `∇f = [∂f/∂x, ∂f/∂y]`
- **Descente de gradient** : algorithme fondamental du ML
  - On part d'un point, on calcule le gradient, on fait un petit pas dans la direction opposée (vers le bas)
  - On répète jusqu'à converger vers le minimum

```
Intuition : tu es sur une colline les yeux bandés.
Pour descendre le plus vite, tu tâtes le sol autour de toi
et tu fais un pas dans la direction la plus pentue vers le bas.
```

✏️ Exercices pratiques

Ex 4.1 — À la main :

- Trouver la dérivée de `f(x) = 3x² + 2x + 1`
- En quel point cette fonction atteint-elle son minimum ?

Ex 4.2 — En Python (visualiser une descente de gradient) :

```python
import numpy as np
import matplotlib.pyplot as plt

# Fonction : f(x) = x²
# Dérivée : f'(x) = 2x

def f(x): return x**2
def df(x): return 2*x  # gradient

# Descente de gradient
x = 3.0          # point de départ
lr = 0.1         # learning rate (taille du pas)
history = [x]

for _ in range(20):
    x = x - lr * df(x)  # on descend dans la direction opposée au gradient
    history.append(x)

# Visualisation
xs = np.linspace(-3.5, 3.5, 100)
plt.plot(xs, f(xs), 'b-', label='f(x) = x²')
plt.plot(history, [f(h) for h in history], 'ro-', label='Descente de gradient')
plt.title("Descente de gradient sur f(x) = x²")
plt.legend()
plt.grid(True)
plt.show()

print(f"Minimum trouvé : x = {history[-1]:.6f}")
```

5. Planning suggéré

Semaine	Contenu	Temps
S1	Algèbre linéaire : théorie + Ex 1.1	3h
S2	Algèbre linéaire : Ex 1.2 Python + approfondissement	3h
S3	Statistiques descriptives : théorie + Ex 2.1	3h
S4	Statistiques : Ex 2.2 Python + histogrammes	3h
S5	Probabilités : théorie + Ex 3.1	3h
S6	Probabilités : Ex 3.2 Python + Bayes	3h
S7	Calcul différentiel : théorie + Ex 4.1	3h
S8	Calcul différentiel : Ex 4.2 Python + révisions	3h

> 💡 Si tu as 6h/semaine, tu peux fusionner 2 semaines en 1 et diviser la durée par 2.

6. Ressources recommandées

Ressource	Lien	Pourquoi
3Blue1Brown — Essence of Linear Algebra	youtube.com/3blue1brown	Visualisations magnifiques, idéal pour comprendre intuitivement
Khan Academy — Algèbre linéaire	khanacademy.org	Cours interactifs, exercices guidés
Khan Academy — Statistiques	khanacademy.org	Très progressif, avec quiz
StatQuest (YouTube)	youtube.com/@statquest	Explications simples et drôles des stats et ML

✅ Critères de validation de l'étape 0.1

Tu peux considérer cette étape validée quand tu es capable de :

- [ ] Expliquer ce qu'est un vecteur, une matrice, et faire un produit scalaire
- [ ] Calculer moyenne, médiane, écart-type d'un petit dataset à la main
- [ ] Expliquer intuitivement ce qu'est une probabilité conditionnelle et le théorème de Bayes
- [ ] Expliquer ce qu'est une dérivée et pourquoi la descente de gradient fonctionne
- [ ] Faire tourner tous les exercices Python de cette fiche sans erreur

Phase 0 — Étape 0.1 | Créé le 22/03/2026